

TFG Identification Module

for Embention KAI — Autonomous Target Identification

Author	Taron Sargsyan
Institution	University of Alicante — AI Engineering
Partner	Embention — Vision Department (KAI Platform)
Hardware Target	TDA4VH-Q1 / J784S4EVM (Jacinto 7 — 32 TOPS)
Date	June 2026
Status	Pre-development technical brief

Project Scope

This document covers the technical analysis, architecture decisions, dataset strategy, and 12-week development roadmap for the Few-Shot Identification Module of the Autonomous Target Engagement (ATE) system on Embention's KAI drone platform. The identification layer is the sole TFG scope — the upstream YOLO detection module is explicitly out of scope.

1. Technical Analysis

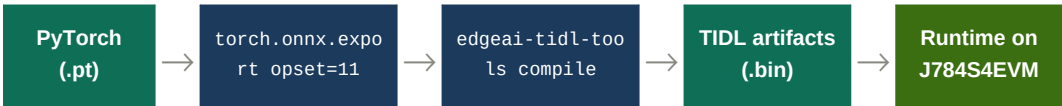
1.1 Hardware Reality Check

The target SoC is the **TDA4VH-Q1** on the **J784S4EVM** (Jacinto 7 family). Key specifications that directly constrain the model design:

Component	Specification	Impact on Model Design
MMAv2 AI Accelerator	4x units — 32 TOPS total at INT8	Post-training INT8 quantization supported and recommended
C7x DSP Cores	4x at 1.0 GHz, 320 GFLOPS FP	Handles layers not offloaded to MMA (pooling, activations)
ARM Cortex-A72	8x at 2.0 GHz, 2 MB L2 per quad	Cosine similarity and gallery lookup run here (cheap)
VPAC / ISP	2x Vision Processing Accelerators	Hardware image preprocessing at zero model budget cost
Security HW	Secure boot, encrypted storage, crypto acceleration	Model IP protection built-in — no extra work required

1.2 TIDL Deployment Chain — Critical Finding

The deployment pipeline is NOT standard ONNX Runtime. TI uses a custom fork (**TexasInstruments/onnxruntime**) that adds **TIDLCompilationProvider** and **TIDLExecutionProvider**. The standard PyPI onnxruntime package does not include these. The full chain:



Architecture constraint: TIDL does not support all ONNX operators. Transformer decoder layers, standalone TopK, and attention mechanisms are unsupported or partially supported on this SDK. ViT, DETR, and all attention-heavy architectures are ruled out. Use CNN backbones, opset 11 only.

No hardware required for compilation. edgeai-tidl-tools compiles models on x86 Linux and supports host emulation mode for correctness testing. The J784S4EVM is only needed for real latency benchmarking (Week 9). Windows users: use WSL2 (Ubuntu 22.04, kernel ~5.15) or TI Edge AI Studio (cloud, browser-based, no installation).

1.3 Dataset Landscape

All datasets are publicly available on Roboflow Universe. Verify each license before use. MIT and CC-BY are safe; anything else must be flagged. Combined estimate after deduplication: **8,000–12,000 labeled crops** — sufficient for Phase 1 pre-training.

Dataset	Images	Key Classes	Perspective	Priority
Military Vehicle Recognition (Roboflow)	~2,000	Tank, APC, bomber, soldier	Drone aerial — Russo-Ukrainian War	Primary
Military Tanks (MilitaryTanks)	5,540	Tank variants	Mixed, YOLO-compatible	Primary
Military Vehicle Detection (oleksandr-tara)	~1,500	16 classes: tank, SAU, RSZO, APC, truck, plane, person	Ground + aerial	Secondary
Military Classification (R. Kaharlytskyi)	1,191	Russian military vehicles	Ground-level	Secondary

1.4 Recommended Architecture

Approach	Verdict	Reason
Prototypical Networks	SELECTED	Single forward pass per crop; clean ONNX export; gallery is a .npy file — no recompilation needed when adding new target classes
Siamese Networks	Rejected	Two-branch graph causes TIDL graph partitioner issues; quadratic inference cost per reference image at runtime
Triplet Networks	Rejected	Complex mining logic at training time with no runtime benefit over Prototypical Networks on this task
Fine-tuning a head	Rejected	Requires model recompilation and retraining for every new target class — not operationally viable in the field

Backbone: MobileNetV3-Small

Selected for TIDL compatibility. Uses Conv, BN, Hardswish, and SE blocks — all supported ONNX operators on TIDL. If Hardswish triggers a compilation error on your SDK version, switch to **MobileNetV2** (ReLU6 only, universally supported) at approximately 1% accuracy cost. Embedding dimension: **128**.

Training Strategy — Two Phases

Phase	Method	Loss	Output	Week
Phase 1	Supervised classification on all available data	Cross-Entropy	Domain-adapted backbone weights	Week 3
Phase 2	Episodic N-way K-shot Prototypical Network training	Cosine CE + learnable temperature tau	backbone.onnx + gallery.npy	Weeks 5–6

ONNX Export Path

```
# Export backbone only – strip the classifier head
backbone = model.features
dummy = torch.randn(1, 3, 224, 224)
torch.onnx.export(
    backbone, dummy, "backbone.onnx",
    opset_version=11,
    input_names=["image"], output_names=["embedding"]
)
# Compile on x86 Linux / WSL2 with edgeai-tidl-tools:
# python3 onnxrt_ep.py --config config_backbone.yaml
# Prototype gallery generation (from backbone.onnx, not .pt):
# python3 scripts/generate_gallery.py \
# --model exports/backbone.onnx \
# --input_dir refs/ --output gallery.npy
```

1.5 Risks and Mitigations

Risk	Prob	Impact	Mitigation
TIDL operator incompatibility (MobileNetV3 Hardswish)	Medium	High	Validate on Monday with Edge AI Studio. Fallback: MobileNetV2. Max 2 hrs debugging before switching.
Dataset domain gap (ground photos vs drone top-down)	High	Medium	Prioritise drone-captured Russo-Ukrainian War dataset. Apply perspective warp augmentation heavily.
N=1 accuracy too low to be useful in isolation	High	Low	Expected behavior. Report at N=1, 3, 5, 10, 50. Let Embention choose their operational N.
No EVM access for latency benchmarking	Medium	Low	TI Edge AI Studio gives approximate latency. Frame Hito 3 as "estimates via host emulation, pending hardware validation."
edgeai-tidl-tools segfault on Linux kernel 6.8+	Medium	Medium	Use WSL2 Ubuntu 22.04 (kernel ~5.15) or TI Edge AI Studio. Validate in Week 1 before WSL2 setup.
Camouflage / occlusion not in public datasets	Medium	Medium	Generate synthetic camouflage with Albumentations: composite vehicle silhouettes onto random terrain backgrounds.

1.6 Open Questions for Embention

Question	Needed by	Reason
Which Processor SDK version is deployed on the KAI platform?	Week 1 (urgent)	Determines the edgeai-tidl-tools git tag for WSL2 local setup. Wrong tag produces silent bad artifacts.
Is J784S4EVM available for SSH access during the TFG?	Week 9	Required for real latency numbers in Hito 3. Cloud Studio provides approximations only.
Does KAI consume raw ONNX or pre-compiled TIDL artifacts with a specific directory structure?	Week 10	Determines the integration glue code required for the final demo pipeline deliverable.

2. Development Roadmap — 12 Weeks

Month 1 — Foundation

Week 1

Environment & TIDL Smoketest

Milestone: Confirmed backbone compiles on TIDL

- Monday — Upload MobileNetV3-Small ONNX to TI Edge AI Studio. Confirm compilation passes. If Hardswish fails, switch to MobileNetV2 today.
- Tuesday — Architecture decision locked. Export backbone.onnx with opset=11. Run onnxruntime CPU inference to confirm output shape.
- Wednesday — Download 4 Roboflow datasets. Verify licenses (MIT / CC-BY). Manual inspection of ~50 images per dataset. Class distribution bar charts.
- Thursday — Literature: Snell et al. (2017) Prototypical Networks + Koch (2015) Siamese Networks. Write 1-page architecture justification document.
- Friday — Set up repo scaffold. Write YOLO integration stub with fake embeddings. Confirm demo.py runs end-to-end with random gallery.

Deliverable: backbone.onnx that compiles on TIDL + repo scaffold + architecture justification doc

Week 2

Datasets & Augmentation Pipeline

Milestone: Training-ready dataset with full augmentation

- Set up WSL2 (Ubuntu 22.04) and clone edgeai-tidl-tools at the SDK version tag confirmed by Embention.
- Build torch.utils.data.Dataset supporting both standard classification batches (Phase 1) and episodic N-way K-shot tasks (Phase 2).
- Augmentation pipeline: HSV jitter, perspective warp (altitude simulation), Gaussian noise, JPEG artifacts, random crop and flip.
- Merge all Roboflow datasets into a unified stratified split: 80% train / 10% val / 10% test, no class leakage.
- Visualise augmented samples and confirm class balance. Log mean and std per channel for normalisation.

Deliverable: Unified Dataset class + augmentation config + split statistics notebook

Week 3

Phase 1 Training — Classification Baseline

Milestone: >80% top-1 accuracy on validation split

- Fine-tune MobileNetV3-Small (ImageNet pretrained) on all military vehicle classes using Cross-Entropy loss.
- Training config: AdamW optimiser, cosine LR decay (1e-3 to 1e-5), 20 epochs, batch size 64, mixed precision.
- Log confusion matrix and per-class accuracy every 5 epochs. Identify problematic class pairs (tank vs APC vs SAU).
- Generate t-SNE visualisation of the 128-dim embedding space at epoch 20 to verify class cluster separation.
- Save the best checkpoint. This backbone is the starting point for all Phase 2 work.

Deliverable: Trained backbone checkpoint + confusion matrix + t-SNE plot

Week 4**Baseline Evaluation & Hito 1 Report***Milestone: Hito 1 deliverable submitted*

- Evaluate Phase 1 backbone on held-out test set: top-1 accuracy, ROC curves per class, macro and weighted F1.
- Verify t-SNE embedding space shows separable clusters. If not, diagnose backbone training before proceeding to Phase 2.
- Document classification head performance as the "standard supervised baseline" for comparison against few-shot in Hito 2.
- Write Hito 1 document: state of the art review, architecture justification, dataset analysis, baseline results.

Deliverable: Hito 1 report draft + full evaluation metrics baseline**Month 2 — Few-Shot Identification Module****Week 5****Prototypical Network Training***Milestone: Episodic training running, initial few-shot accuracy measured*

- Implement episodic training loop: randomly sample N-way K-shot tasks from training set each iteration.
- Prototype computation: mean of K support embeddings per class, L2-normalised to unit sphere.
- Loss: cosine cross-entropy with learnable temperature parameter tau (initialise at 10).
- Initial evaluation: 5-way 1-shot and 5-way 5-shot accuracy on validation episodes.
- Compare embedding space t-SNE before and after Phase 2 training. Expect tighter intra-class clusters.

Deliverable: Prototypical Network training loop + initial few-shot accuracy table**Week 6****N-way K-shot Ablation Study***Milestone: Accuracy vs N trade-off curve ready for Embention*

- Systematic ablation: test 5-way at K = 1, 3, 5, 10, 20, 50 reference images. Report mean accuracy over 1,000 random episodes.
- Backbone comparison on 5-way 5-shot: MobileNetV3-Small vs MobileNetV2.
- Embedding dimension comparison on 5-way 5-shot: 64 vs 128 vs 256.
- Produce the accuracy/latency/memory trade-off table that will appear in the final report and Embention presentation.
- Finalise backbone + embedding dimension choice based on results. Freeze the model configuration.

Deliverable: Ablation tables + trade-off analysis — core TFG result

Week 7 Embedding Generation Script

Milestone: *generate_gallery.py* tested and documented

- Write scripts/generate_gallery.py: accepts a reference image folder per class, runs the ONNX backbone, computes mean prototype embeddings, saves gallery.npy.
- CLI: --model exports/backbone.onnx --input_dir refs/ --output gallery.npy --n_shots K
- Validate: regenerate gallery at K=1, 5, 10 and confirm identification accuracy matches Phase 2 evaluation numbers.
- Critical: the gallery must be generated from backbone.onnx (not the .pt file) to match the on-device runtime path.
- Write docstring and add usage section to README.md.

Deliverable: generate_gallery.py script + documented README section

Week 8 Robustness Testing

Milestone: *Failure mode analysis complete, limitations section written*

- Degrade reference images: Gaussian blur (sigma 2, 4), JPEG compression (q=50, 25), partial occlusion (25%, 50% crop).
- Degrade query images: fog overlay, low-light gamma (0.3), rain augmentation, motion blur.
- Measure accuracy at N=1 and N=5 under each perturbation. Identify worst-case class confusion pairs.
- Test test-time augmentation (TTA): average embeddings from 5 augmented crops of the same query. Measure accuracy gain vs cost.
- Write limitations section for final report: what the system cannot handle reliably.

Deliverable: Robustness analysis tables + identified failure modes + limitations draft

Month 3 — Optimisation & Demo

Week 9 ONNX Export & TIDL Compilation

Milestone: *backbone.onnx* compiled and latency measured

- Export final backbone checkpoint to ONNX opset=11. Verify on CPU onnxruntime: outputs match PyTorch within 1e-4.
- Compile with edgeai-tidl-tools (TIDLCompilationProvider). Generate TIDL artifacts directory.
- Profile latency via TI Edge AI Studio or J784S4EVM if available. Frame budget: 33 ms total at 30 fps. YOLO ~12 ms, backbone target <15 ms.
- If latency exceeds budget: apply TIDL post-training INT8 quantization, or reduce input from 224x224 to 160x160.
- Document the accuracy delta between FP32 and INT8 (expect <2% drop at standard quantization settings).

Deliverable: Compiled TIDL artifacts + latency profile + quantization comparison table

**Week
10****End-to-End Demo Pipeline***Milestone: demo.py runs on a video file with annotated output*

- Integrate pre-trained YOLOv8n (Ultralytics, yolov8n.pt) as the upstream detector. Do not train YOLO — it is out of TFG scope.
- Write scripts/demo.py: video input → YOLO detection → crop → backbone.onnx embedding → cosine vs gallery.npy → annotated frame output.
- Add confidence threshold: suppress labels below 0.70 similarity score. Display class name + score on bounding box.
- Test on drone footage of vehicles (YouTube, open datasets, or simulator screenshots).
- Record a 60-second demo video for inclusion in the final Embention presentation.

Deliverable: scripts/demo.py + 60-second demo video recording**Week
11****Code Cleanup & Documentation***Milestone: Repo reproducible from a fresh clone with one command*

- Enforce final repo structure: src/model/, src/data/, src/inference/, scripts/, notebooks/, exports/, checkpoints/.
- Write README.md: installation steps, dataset setup commands, training command, evaluation command, demo usage.
- Add docstrings and type hints to all public functions. Remove dead code and debug prints.
- Verify reproducibility: fresh conda env + pip install -r requirements.txt + single training command reproduces final numbers within 0.5%.
- Write final LaTeX memory draft: abstract, methodology, experiments, trade-off analysis, conclusions.

Deliverable: Clean repo + requirements.txt + README.md + LaTeX memory draft**Week
12****Final Report & Presentation***Milestone: TFG submitted; Embention presentation delivered*

- Finalise LaTeX memory: abstract, introduction, state of the art, methodology, experiments, limitations, conclusions, references.
- Prepare Embention presentation: 12-15 slides — problem statement, approach, ablation tables, demo screenshots, latency analysis, limitations, next steps.
- Verify all four TFG success criteria from the proposal: viability demo, trade-off analysis, ONNX export, documentation quality.
- Rehearse 5-minute live demo: laptop + pre-recorded video as backup.
- Submit TFG memory and repository link.

Deliverable: TFG submission + Embention slides + demo video